Seeing to Drive on a Single Board: Attention-Free Distilled Depth on a Raspberry Pi 5 + Hailo-8L for Onboard Obstacle Avoidance, Planning and SLAM

Itai M.

Abstract—**Small ground robots can afford one camera and a few watts of compute. Monocular depth foundation models such as Depth Anything V2 predict excellent relative depth, but they are vision transformers sized for GPUs. We take one to a Raspberry Pi 5 with the 13-TOPS, integer-only Hailo-8L accelerator and build a complete onboard autonomy stack on it. We first show by measurement why transformer depth breaks on this chip: attention runs in int8, and at 449 tokens 91 % of softmax weights fall below one quantisation step while carrying 17–33 % of the attention mass. We therefore distil Depth Anything V2-Large into an attention-free ResNet-34 U-Net (24.5 M parameters). On the real Hailo-8L it runs at 27.1 FPS at 288×512, its int8 output matches its float twin at Pearson 0.99993, and on 400 held-out simulated frames with metric ground truth it is the most accurate depth network on the chip (AbsRel 0.269 against 0.354–0.530, including the vendor's own build); on 1,000 stereo-verified points of a real outdoor drive it is not resolvably different from the vendor's Depth Anything V2-Small; its simulated advantage on obstacles is not shown there (a post-hoc, exploratory lean; Holm p 0.046–0.14 by test family). We range obstacles by ground-contact geometry, which needs only the camera height, and run a hit-count map, hybrid-A* planning and ground-plane SLAM on the Pi's CPU in parallel with the NPU. In a real-time hardware-in-the-loop evaluation in a Gaussian-splat reconstruction of a real yard, the deployed stack reaches the goal without contact in 7.9 of 10 scenes with simulator pose, 7.7 on its own pose (ten seeds each) and 8.2 with the real Pi and Hailo-8L in the loop, against 9.2 for the same stack fed exact obstacles.**

I. INTRODUCTION

A rover that drives through a yard of chairs, cones, crates and people must know where things are. Range sensors answer that at a cost in money, power, mass and blind spots: a planar LiDAR misses a table top and a low curb, and a palm-sized car has no room for a stereo baseline. A single RGB camera sees all of these, but gives no distance directly.

Monocular depth has changed character in the last three years. Networks trained on tens of millions of images, from MiDaS [1] to Depth Anything [2], Depth Anything V2 [3], Depth Pro [4] and Depth Anything 3 [5], now predict relative depth sharp and stable enough to drive with. They are vision transformers sized for data-centre GPUs: Depth Anything V2-Large has 335 M parameters and takes 175 ms per frame even on the RTX 5090 laptop GPU used in this work. A small robot has a few watts.

We ask how much of that capability survives on the cheapest serious edge platform available today, a Raspberry Pi 5 with the 13-TOPS Hailo-8L of the Raspberry Pi AI Kit [6], and what a rover can do with it. The answer is not a compilation exercise. The accelerator executes every layer in integer arithmetic, attention included, and both our int8 build of Depth Anything V2-Small and the vendor's lose large parts of the scene. Even a faithful depth map is only relative, so it cannot by itself tell a planner that a chair is 1.2 m away. And a car needs a map, a planner, a controller and an estimate of its own position within the same board and control period.

Contributions.

1) A measured explanation of why transformer depth fails on the Hailo-8L (int8 cannot represent diffuse attention at a few hundred tokens), and an attention-free student distilled from Depth Anything V2-Large whose int8 output equals its float output on the chip (Sections III and IV).
2) A head-to-head benchmark of every depth network we could run on the Hailo-8L, on 400 held-out simulated frames with metric ground truth and 100 real robot frames: on the simulated frames the student is the most accurate on the chip, and it is faster than both Depth Anything V2-Small builds (Section V). On 1,000 stereo-verified points of a real outdoor drive it is not resolvably different from the vendor's Depth Anything V2-Small (Section IX).
3) Scale-free metric ranging from relative depth by ground-contact geometry, and a complete onboard stack on the Pi (map, hybrid A*, pure pursuit, ground-plane SLAM with wheel odometry and a slip guard) with its measured real-time budget (Sections VI and VII).
4) A real-time hardware-in-the-loop evaluation with the real board, an explicit error budget from oracle to camera-only odometry, and the negative results reported with the same rigour as the positive ones (Section VIII).

II. RELATED WORK

Monocular depth. MiDaS [1] introduced the scale- and shift-invariant loss that makes relative depth transfer zero-shot, DPT [7] moved it to a vision transformer, and Depth Anything [2] scaled pseudo-labelling to 62 M images. Depth Anything V2 [3] trains a large teacher on synthetic ground truth and distils it into 25–335 M-parameter students. Metric models condition on or estimate the camera [8], [9], [4]; Depth Anything 3 [5] predicts consistent geometry from any number of views; Distill Any Depth [10] shows that distillation choices alone strengthen a student. All are transformers, and none targets an integer-only accelerator.

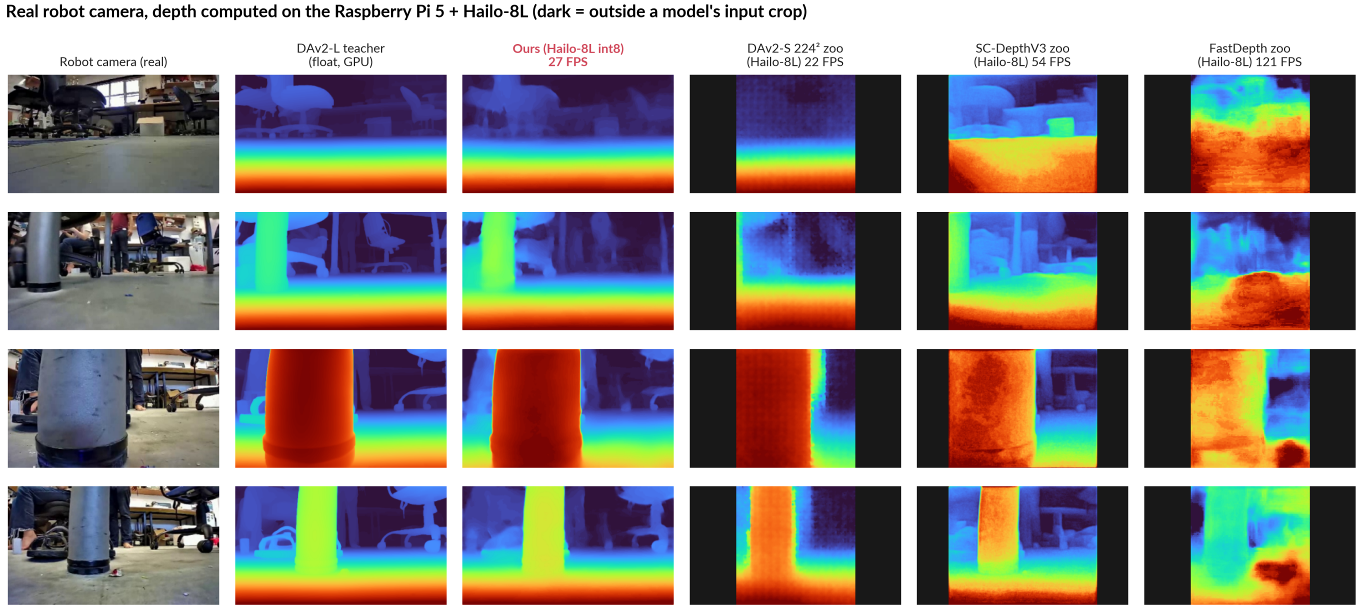

Real robot camera, depth computed on the Raspberry Pi 5 + Hailo-8L (dark = outside a model's input crop)
Robot camera (real)
DAv2-L teacher (float, GPU)
Ours (Hailo-8L int8) 27 FPS
DAv2-S 224² zoo (Hailo-8L) 22 FPS
SC-DepthV3 zoo (Hailo-8L) 54 FPS
FastDepth zoo (Hailo-8L) 121 FPS

Fig. 1. Depth computed *on the Hailo-8L* for real robot frames, against the float Depth Anything V2-Large teacher on a GPU. Our attention-free student keeps the teacher's structure at 27 FPS; the int8 transformer builds (ours and Hailo's Model Zoo) lose it, and the small self-supervised zoo models are coarse. Dark areas lie outside a model's input crop.

Depth on small computers. FastDepth [11] runs a pruned MobileNet encoder-decoder on a Jetson; Lite-Mono [12] and SC-DepthV3 [13] learn from video without labels. FastDepth and SC-DepthV3 are the two depth models Hailo ships for the Hailo-8L, and both appear in our benchmark. AsyncMDE [14] amortises a foundation model over time on a Jetson AGX Orin. Vision transformers are hard to quantise because post-softmax activations are long-tailed; FQ-ViT [15], PTQ4ViT [16] and RepQ-ViT [17] need quantiser support that a fixed NPU toolchain may not offer.

Navigating with predicted depth. MonoNav [18] fuses transformer depth into a reconstruction to fly a Crazyflie, with depth computed off-board. MonoMPC [19] uses depth as context for a learned collision model. Finke et al. [20] fine-tune Depth Anything V2 for four cameras and run an RL policy on a Jetson AGX Orin at 10 Hz. Learned policies [21], [22] and imperative planners [23], [24] map images to paths. We keep planning classical (hybrid A* [25], pure pursuit [26]) so that every failure can be attributed to perception, planning or localisation, and run all of it on a board an order of magnitude cheaper than an AGX Orin.

III. Why Transformer Depth Breaks on the Hailo-8L

The Hailo-8L runs every layer with 8-bit weights and activations, including the matrix products and softmax of attention. We exported Depth Anything V2-Small at 224×392 (a 16×28 grid of 14-pixel patches plus a class token: 449 tokens), calibrated it on domain images and compiled it. On real robot video the result looked right (median Pearson 0.99 to the float model). On the yard renders it erased obstacles on a third of held-out frames, dissolving a crate or a person into the floor (Figure 1).

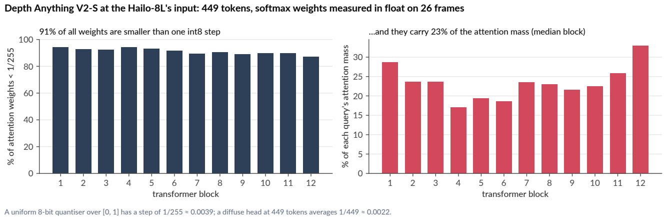


Fig. 2. Float Depth Anything V2-Small at the Hailo-8L's 449-token input: per block, the share of softmax weights below one uniform int8 step (left) and the share of attention mass they carry (right).

A head that spreads attention over many tokens produces softmax weights near $1/449 \approx 0.0022$, while one step of a uniform 8-bit quantiser over $[0, 1]$ is $1/255 \approx 0.0039$. Such weights round to zero or to one step. Measured in float Depth Anything V2-Small at the chip's input on 26 held-out frames (Figure 2), 91 % of all softmax weights are below one int8 step, and they carry 17–33 % (median 23 %) of each query's attention mass, most of it in the last block, which feeds the depth head. The available remedies did not close the gap: re-calibration plus the compiler's quantisation fine-tuning moved the share of frames with Pearson below 0.8 only from 48 % to 37 %, sensor-noise calibration models left 30–39 % broken, and Hailo's own Model Zoo build shows the same blockiness and object loss. The failure is structural; removing attention fixes it.

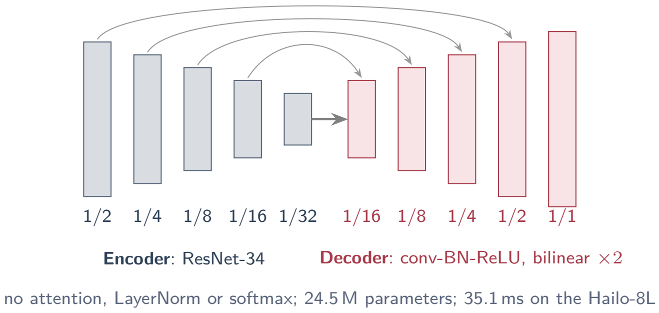

1/2
1/4
1/8
1/16
1/32
1/16
1/8
1/4
1/2
1/1
Encoder: ResNet-34
Decoder: conv-BN-ReLU, bilinear ×2
no attention, LayerNorm or softmax; 24.5 M parameters; 35.1 ms on the Hailo-8L

Fig. 3. The attention-free student: a U-Net from 288×512 RGB to disparity at full resolution (grey: skip connections).

IV. An Attention-Free Student

We distil Depth Anything V2-Large into a network with no attention, no LayerNorm and no softmax: a ResNet-34 encoder [27] with ImageNet initialisation and a U-Net decoder [28] of convolution, batch normalisation, ReLU, bilinear 2× upsampling and skip concatenation. It has 24.5 M parameters, almost exactly Depth Anything V2-Small's 24.8 M, so the comparison isolates architecture from capacity. It maps 288×512 RGB, the robot camera's aspect, to relative disparity at full resolution.

Training. The loss is the scale- and shift-invariant L1 of MiDaS plus four-scale gradient matching, so only the shape of depth is supervised. v1 is distilled from teacher labels on 7,230 simulated point-of-view frames and 720 real frames, with three scenes (crates, people, barrels) and the real robot video held out. v3 fine-tunes v1 for eight epochs on *rendered* ground-truth disparity for simulated frames (13,902 simulated + 2,024 real), which reduces the teacher-inherited range bias on simulated props from about $-30\,\%$ to $-7\,\%$. v5, the deployed student, weights the loss $\times 4$ on rows below the horizon, where obstacles meet the floor; it cuts spurious near-floor hits from 13.7 % to 10.2 %.

On the chip. The student compiles with normalisation folded into the HEF. Replayed on the real chip against its own float weights on 500 frames, v3 reaches Pearson 0.99995 at the median (5th percentile 0.99897) and v5 0.99993 (0.99569). The compiler's emulator had reported a correlation of -0.41 for this graph, a false alarm: an NPU build should be judged by replay on the chip. The student takes 35.1 ms on the NPU and 37.0 ms per frame end to end, 27.1 FPS at batch 1 (59 FPS at batch 8), against 455 ms on the Pi's four CPU cores.

V. On-Chip Benchmark

Protocol. 400 simulated frames from 36 episodes of the three held-out scenes, with the renderer's metric depth, and 100 held-out real robot frames without ground truth. Every Hailo-8L model runs on the chip with its outputs read back. Each model sees a centre crop at its own aspect; predictions are aligned to ground-truth disparity by per-frame least-squares scale and shift [1]. We report AbsRel and δ_1 on all valid pixels and on *obstacle* pixels: pixels within 3 m that, back-projected with the true pose, lie on a prop's collider. The obstacle region removes the renderer's floor speckle and measures what the planner reads. On real video the metric is Pearson correlation to the teacher.

TABLE I
Depth on held-out frames (400 sim frames with metric GT; r: median Pearson to Depth Anything V2-Large on 100 real frames). Bold: best on the chip.

Model	AbsRel↓		δ_1 ↑	r ↑	Speed
	all	obst.	all	real	
On the Hailo-8L (int8), FPS on the Pi					
Ours, student v5	0.269	**0.197**	0.669	0.996	27.1
Ours, student v3	**0.269**	0.204	0.674	0.995	27.1
DAv2-S, Hailo zoo	0.354	0.334	0.593	0.979	21.6
DAv2-S, our int8	0.477	0.330	0.423	0.982	16.6
SC-DepthV3, zoo	0.414	0.352	0.506	0.950	53.6
FastDepth, zoo	0.530	0.767	0.344	0.709	121.3
Float on the RTX 5090 GPU, ms per frame					
DAv2-L (teacher)	0.284	0.144	0.643	1.000	175 ms
DAv2-S	0.283	0.150	0.651	0.999	39 ms
DA3-Small	0.315	0.172	0.611	0.988	42 ms
DA3-Base	0.304	0.188	0.616	0.985	40 ms

Results (Table I and Figure 4). On the chip the student is in a different class: overall AbsRel 0.269 against 0.354 for Hailo's Depth Anything V2-Small, 0.414 for SC-DepthV3, 0.477 for our Depth Anything V2-Small build and 0.530 for FastDepth, and 0.197 against 0.330–0.767 on obstacle pixels. On real video it tracks its teacher at $r = 0.996$, where the vendor's Depth Anything V2-Small reaches 0.979 and FastDepth 0.709. Speed is not traded for this: at 27.1 FPS it is faster than both Depth Anything V2-Small builds (21.6 and 16.6 FPS) while processing 2.9× and 1.7× more pixels. Against float foundation models the student gives up a few points on obstacles (the teacher reaches 0.144) but is better overall, because it was fine-tuned on rendered ground truth of this domain; Depth Anything 3 is not better than Depth Anything V2 here. The simulated frames come from one reconstructed yard; the real frames show that the fine-tuning did not damage real-world behaviour, but agreement with a teacher is not accuracy.

VI. From Relative Depth to a Metric Scan

The student's disparity is $d = a/Z + b$ with unknown, drifting a and b. Instead of learning metric depth we take metre scale from the camera height h. For a level camera with focal length f_y and principal row c_y, a floor point at image row $v > c_y$ lies at

$$Z_{\text{floor}}(v) = \frac{f_y\, h}{v - c_y}. \tag{1}$$

An obstacle meeting the floor at row v_c is at $Z_{\text{floor}}(v_c)$, whatever the network's scale. The network only has to say where in each column the floor ends, a question about the shape of the depth map. The image columns become a 69°, 4-metre "laser scan".

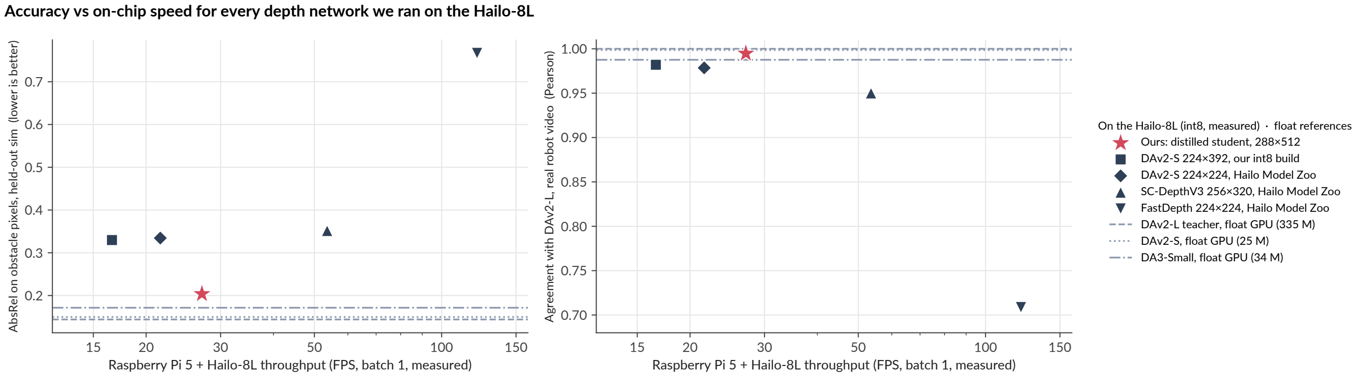


Fig. 4. Accuracy against measured on-chip throughput on the Raspberry Pi 5 + Hailo-8L (batch 1). Left: AbsRel on obstacle pixels. Right: agreement with Depth Anything V2-Large on real robot video. Horizontal lines: float GPU models that cannot run on the Pi at this rate.

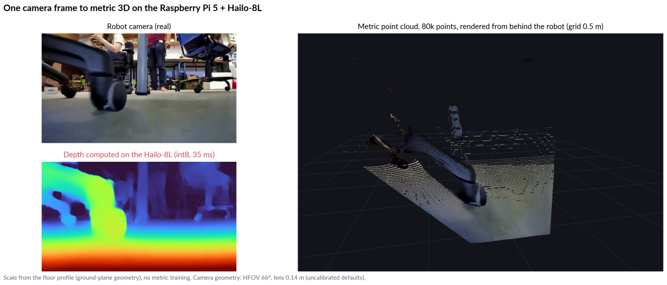

One camera frame to metric 3D on the Raspberry Pi 5 + Hailo-8L
Robot camera (real)
Depth computed on the Hailo-8L (int8, 35 ms)
Metric point cloud, 80k points, rendered from behind the robot (grid 0.5 m)
Scale from the floor profile (ground-plane geometry), no metric training. Camera geometry: HFOV 66°, lens 0.14 m (uncalibrated defaults).

Fig. 5. A real robot frame lifted to metric 3D on the Pi: chip depth, scale from the floor profile, flying pixels removed.

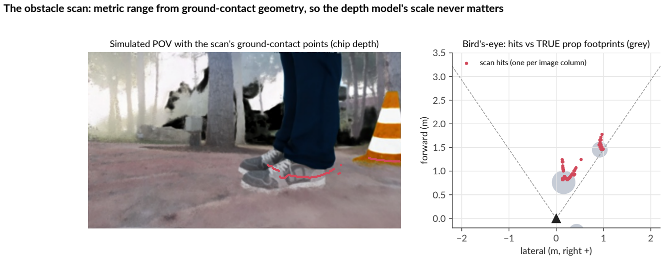


Fig. 6. The scan on a held-out frame from Hailo-8L depth: one ground-contact hit per column (left) and in bird's-eye view against the true prop footprints (right).

The scan. Per row, the 25th percentile of disparity over the central 80 % of columns models the floor $f(v)$, made monotone from the horizon down. For a pixel, $(d-b)/(f(v)-b) = Z_{\text{floor}}/Z$ is scale-free; a pixel at least $H_{\min} = 5\,\text{cm}$ above the floor at its range exceeds $1/(1 - H_{\min}/h)$, and the first run of five such rows in a column is the obstacle's contact. A second, profile-free detector flags vertical faces, where the normalised disparity slope $\partial_v d\,(v - c_y)/(d - b)$ drops from ≈ 1 (floor) to ≈ 0 (face). Following the contact upward gives an obstacle's height; only hits at least 6 cm tall are mapped, which cut phantom hits from 2,376 to 75 on six recorded episodes. Two fixes came from failure analysis: anchoring the floor profile at the horizon (a chair seat above the camera had become "floor", making a tube at 0.68 m read 1.16 m), and a detector for floating overhangs such as seats and table tops (Figure 6). The same profile is a ruler for every pixel, $Z = Z_{\text{floor}}(v)(f(v) - b)/(d - b)$, which gives a metric point cloud from one frame without metric training (Figure 5); pixels at depth discontinuities are dropped. Per pixel it is less robust than the contact scan (median error -2 to $+8\,\%$ on obstacle pixels, but a long tail at object edges), so we use it for overhangs and visualisation, not for ranging.

Map and limit. Hits accumulate in a 5 cm world grid; a cell becomes an obstacle after three hits and if it holds at least 30 % of the hits of the densest cell within 0.2 m. The remaining perception error is low structure: at the deployed 5 cm threshold the scan sees prop sides 5.5 cm (ground-truth depth) and 5.1 cm (student) inside the true footprint. Lowering the threshold to 2 cm closes the gap for ground-truth depth ($-0.5\,\text{cm}$) but not for the student (+2.5 cm), whose floor-to-base boundary is blurred.

VII. Onboard Autonomy

Planning. A hybrid A* [25] over rear-axle poses (0.1 m cells, 72 headings, ten 0.25 m primitives forward and backward) checks a three-circle footprint against the sensed map and a prior drivability map of the yard. Its heuristic is the larger of an obstacle-aware 2D geodesic distance and a precomputed table of the obstacle-free non-holonomic cost of the lattice; without the table, a goal behind the car flooded 40,000 expansions. Primitives are planned at 75 % of steering lock (1.17 m radius), because the car turns wider than its bicycle model and plans at full lock saturated the follower. A cascade relaxes the clearance margin from 0.10 to 0.06 m when a search fails and widens it to 0.20 m when that costs at most 1.6× the length. Cells the car already overlaps are passable only within 0.3 m of the start, so it can back out of a contact without planning through a touched crate face. Searches run in a worker process: 110 ms at the median and 547 ms at the 90th percentile on the Pi.

Control and safety. Pure pursuit tracks one direction segment at a time. The car re-plans when the path is blocked, when it leaves the path, or when it is wedged (throttle without progress

TABLE II

Self-localisation alternatives, offline on 50 recorded 35-s episodes (absolute trajectory error, m).

Odometry source	median	p90
Ground-plane VO (camera only)	0.42	1.32
VO translation + gyro heading	0.36	1.04
Encoder + VO heading	0.22	1.18
Encoder + Ackermann heading from steering	0.25	0.61
Encoder + gyro	**0.08**	**0.19**

for 1.5 s, which also places a "bump" obstacle at the bumper). Every reverse passes the Traxxas ESC's brake-then-neutral gate, which the simulator models.

SLAM. About 400 floor features within 1.5 m are tracked with forward–backward Lucas–Kanade [29], lifted to metric floor points with the same camera-height geometry, and fit by a RANSAC 2D rigid transform. Wheel odometry (encoder and gyro) enters directly. A slip guard declares the car still when the rotation-compensated floor flow is below 2 px while the encoder claims at least 2 cm per frame. Offline on 50 recorded episodes (Table II) an IMU alone barely helps; the wheel encoder is what makes own-pose navigation work. In the closed loop, with encoder and gyro, the onboard estimate stays within 0.13 m of the truth at the median and 0.99 m at the 90th percentile over ten seeds (Figure 7).

Real-time budget. With the deployed code on the real board the NPU takes 35.0 ms per frame and the CPU stages (scan, map, SLAM, navigation) 46.7 ms at the median (64.1 ms at p90), overlapped (Figure 8). On real 1280×720 video the standalone runner reaches 16.5 Hz with SLAM and 27 Hz without it, limited by the NPU. Over 1000 episodes and 28 h the NPU time did not drift (p99 35.9 ms) and the firmware's throttle register stayed clear. The Hailo-8L is rated at about 1.5 W [6]; the module exposes no power sensor, so we report no measured power.

VIII. Hardware-in-the-Loop Evaluation

Set-up. Isaac Sim with Isaac Lab [30] renders a Gaussian-splat reconstruction [31], [32] of a real yard with a height-field collider. A model of the project's Traxxas drives with ±0.349 rad steering and up to 0.72 m/s; its level camera is 504×280 with a 69.4° field of view, 0.13 m above the ground. Eleven prop types (chairs, a cone, a box, a barrel, a bin, a sign, a crate, two people) carry colliders sized to what the car can hit. Ten frozen scenes put 3–6 m between start and goal; three were never seen by the depth network. Every frame is sent to the robot computer, and each returned command is applied only when it arrives. An episode is *clean* if the goal is reached (within 0.45 m, in 35 s) and no point of the car's real 0.557×0.294 m footprint ever touched a collider. Every arm runs at least five seeds, each jittering the start.

The board in the loop. The first arms ran on the real Pi and Hailo-8L. To run arms in parallel, the identical Pi code also runs on the laptop with replies padded to the Pi's measured round trip and plans slowed to the Pi's speed. The deployed configuration was then re-run on the real board: 8.2 ± 0.4 clean against 7.6 for the laptop twin on the same seeds, with a median round trip of 85 ms (p99 113 ms) over 9,371 replies against the twin's 84 ms.

TABLE III

Clean goals of 10 (mean ± sd over n seeds) and goals reached.

Configuration	n	Clean	Reached
Oracle: exact props, lockstep	5	9.8 ± 0.4	10.0
Real time, exact obstacles	5	9.2 ± 0.4	9.8
Student v3, GT pose	10	7.9 ± 1.0	9.0
Student v5, GT pose	10	7.9 ± 1.1	9.0
Student v3, own pose	10	7.1 ± 0.9	7.4
Student v5, own pose	10	7.7 ± 0.7	8.1
v5, own pose, real Pi + Hailo	5	8.2 ± 0.4	8.4
Camera-only VO (v1, earlier planner)	5	2.6 ± 0.9	3.6

The error budget (Table III and Figure 9). An oracle that knows every prop and plans in lockstep reaches 9.8: a perfect score is feasible. The same real-time stack fed exact obstacles reaches 9.2, so latency, the follower and the ESC gate cost about 0.6. With depth from the Hailo-8L, the deployed stack reaches 7.9 ± 1.1 (9.0 reached): perception costs about 1.3. On its own pose with simulated encoder and gyro it reaches 7.7 ± 0.7; self-localisation costs 0.2 clean goals but 0.9 goals reached. Camera-only odometry reaches 2.6, so a wheel encoder is not optional for this car. With simulator pose most misses reach the goal after a graze (11 of 21); on own pose most are timeouts (18 of 23).

v3 or v5. The floor-weighted v5 replaced v3 on modest evidence, which we state exactly: with simulator pose the two are not resolved (7.9 vs 7.9 at ten seeds), with per-scene differences in both directions; on own pose v5 leads, 7.7 against 7.1, not significantly (permutation test $p \approx 0.17$). It costs nothing on the chip and has the lower obstacle AbsRel.

What remains, and what did not help. The remaining contacts are almost all the car's front corner grazing the side of a cone, a person or a crate that the scan places 5–15 cm too far out: the low-base limit of Section VI. In one scene the goal starts 63° off axis and a crate lies outside the ±34° view, so first plans run through unseen space; this is an exploration problem. More than a dozen interventions moved contacts within the seed noise and are reported as such (Table IV).

One comparison is worth stating plainly: the student drives better than the simulator's own depth buffer (6.8 vs 5.2), whose splat floaters and floor speckle corrupt ground-contact ranging, even though the renderer's arm ran without latency.

IX. A Real Drive: Depth and SLAM against the RealSense

The car was driven by hand through a sunlit park while it recorded every D435 stream (colour, on-chip stereo depth, both infrared images) and the student's Hailo-8L output for every frame: 219 s recorded, of which the SLAM window (first

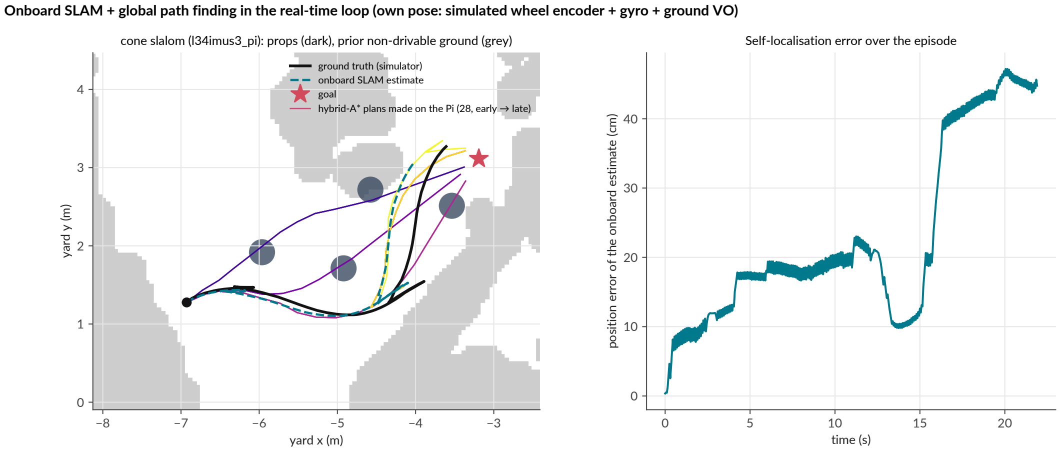


Fig. 7. Onboard SLAM and global planning on the robot's own pose: every hybrid-A* plan the Pi made in one cone-slalom episode, the SLAM estimate against the truth, and the localisation error over time. The car reached the goal despite 0.47 m peak drift.

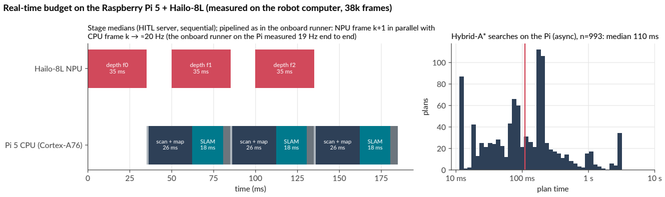


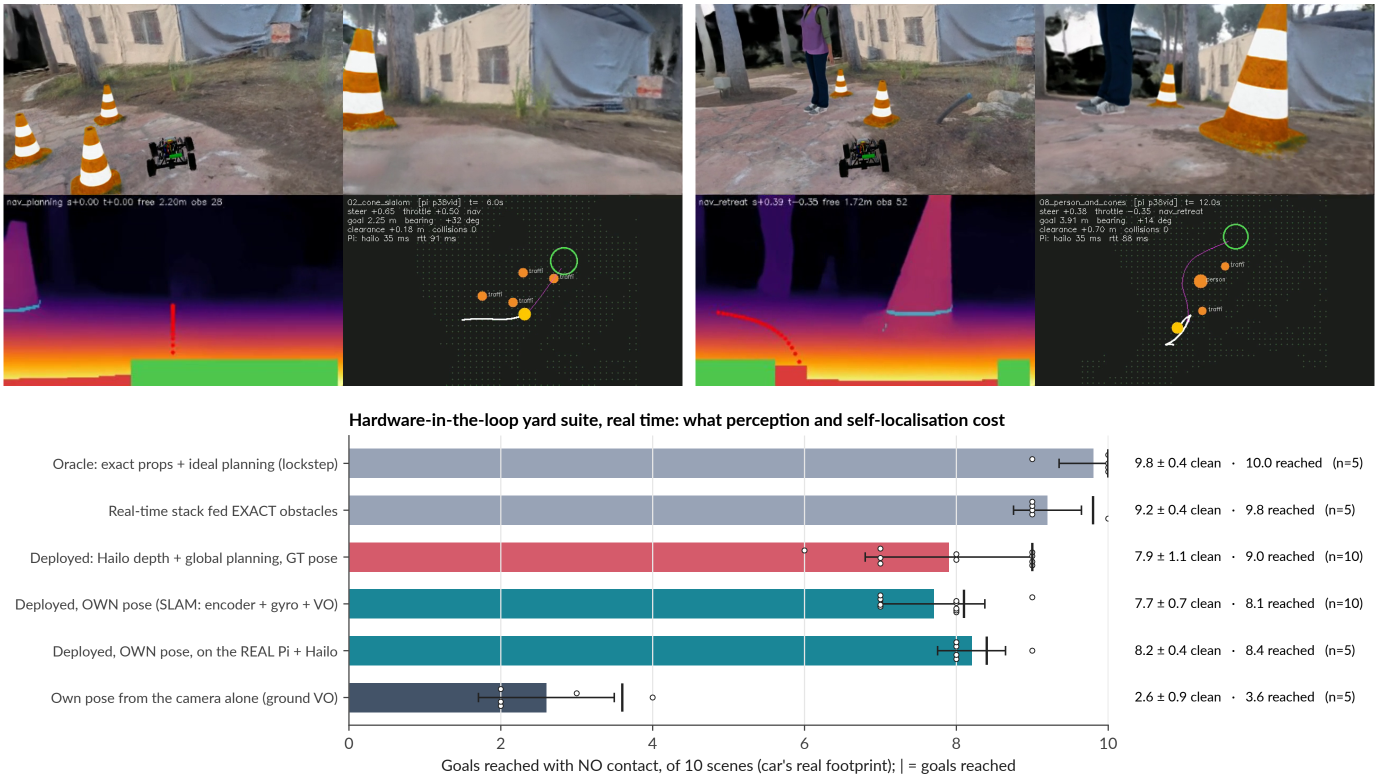
Fig. 8. Per-frame stages on the Pi and how the runner overlaps NPU and CPU (left); hybrid-A* search times (right).

TABLE IV
INTERVENTIONS THAT DID NOT IMPROVE CLEAN SUCCESS ($\geq$5 SEEDS; AGAINST THE BASELINE OF THE TIME ON THE SAME SEEDS).

Intervention	Outcome	Clean
+6 cm skirt on small obstacles	slalom closes	7.2 vs 7.6
Orange-mask cone contacts	slalom timeouts	7.8 vs 7.6
Look-first arc, off-axis goal	neutral, own pose	6.8 vs 7.1
Third planning pass	contacts move	7.8 vs 8.0
Faster follower	drift p90 ×1.6	7.4 vs 7.6
Uniform blocked-path margin	re-plans stall	6.6 vs 7.4
2× camera pixels	no change	2.6 vs 2.6
Renderer depth for the student	floaters, speckle	5.2 vs 6.8

stereo frame to the end) is 216 s. Three defects of the recorder itself were found and worked around before any comparison: a byte shift from the capture tool's own header (recovered losslessly), stereo timestamps that drift ≈8 % against their content (frames re-synchronised from the images; yaw residual median 12.5 ms, p90 31.7 ms; 49 of 388 evaluation frames excluded: 26 sync-suspect, 23 without an IR frame), and a power cut that lost the last seconds of every stream.

Which depth is better. The drive has no lidar, so the reference is agreement between three different matchers on the same infrared pair (D435 ASIC, SGBM, FastFoundationStereo; within 2 % + 1 %/m, 0.3–3 m). FastFoundationStereo's backbone includes Depth Anything V2 features, but as one voter it can only veto points. Points are registered into the colour view with the D435's factory calibration, read after the drive: the nominal values misplaced every point by 14.5 px; on the same points the factory values cut 17 of 19 networks' AbsRel by 10–47 % (stereo unchanged) and align depth with colour edges better on all 20 scorable frames. 1,000 points are frozen once and every monocular method is scored on them, relative models aligned per frame on held-out points (Table V); CIs are frame-cluster bootstraps, re-checked with 10 s blocks as units. The student, live on the chip, reaches AbsRel 0.262 and is not resolvably different from the Hailo model-zoo Depth Anything V2-S (paired difference +0.017, 95 % CI $[-0.060, +0.118]$, $p = 0.70$; $|\log(\hat{z}/z)|$ +0.009, CI $[-0.012, +0.032]$): on these mostly-ground points the interval excludes an advantage as large as the simulator's (0.269 vs 0.354 all-pixel AbsRel on the yard it was fine-tuned on, a different metric), though not on the two-matcher set below. On an obstacle stratum (points $\geq$0.15 m above the floor plane; 319 points, 87 frames; added after the main null), $|\log(\hat{z}/z)|$ leans the simulator's way, ours against the zoo model -0.165, but with blocks exact $p = 0.046$ (Holm 0.046–0.14 by family; exploratory) and -0.123, $p = 0.055$, on the two-matcher set; the median is not lower (0.232 vs 0.240), and with one scale per drive ours is worse there (+0.334, $p = 0.040$, Holm 0.058). Depth Anything V2-L (not on the two-matcher set) and DA3-L are better on obstacles by 0.153 and 0.184. Without Fast-FoundationStereo (a new 1,000-point set on 321 frames) the mean leans our way (-0.061, CI $[-0.132, +0.001]$), carried by the zoo model's blow-ups ($|\log|$: -0.006), so the Depth Anything V2-derived voter does not drive the null. Live, the student is worse than its own chip replay by +0.004 (CI $[0.001, 0.006]$; not on the two-matcher set). It beats the other three edge builds ($p < 0.001$; SC-DepthV3 and FastDepth get a median scale only) and trails its teacher Depth Anything V2-L by 0.085 per frame (CI



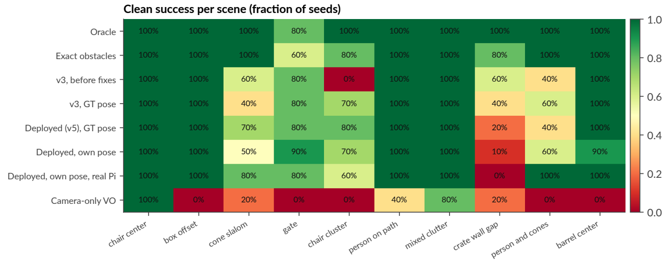
```
nav_planning s+0.00 t+0.00 free 2.20m obs 28
```

```
02_cone_slalom  [pi p38vid]  t=  6.0s
steer +0.65  throttle +0.50  nav
goal 2.25 m  bearing   +32 deg
clearance +0.18 m  collisions 0
Pi: hailo 35 ms  rtt 91 ms
```

```
nav_retreat s+0.39 t-0.35 free 1.72m obs 52
```

```
08_person_and_cones  [pi p38vid]  t= 12.0s
steer +0.38  throttle -0.35  nav_retreat
goal 3.91 m  bearing   +14 deg
clearance +0.70 m  collisions 0
Pi: hailo 35 ms  rtt 88 ms
```

Fig. 9. Top: the deployed stack with the real Pi + Hailo-8L in the loop on its own SLAM pose (chase camera, robot camera, on-chip depth with the commanded arc, bird's-eye map with the SLAM path and current plan); left, cone slalom; right, backing away from a cone seen at close range. Bottom: the error budget, from an oracle to camera-only odometry (dots: seeds; ticks: goals reached).



Clean success per scene (fraction of seeds)

	chair center	box offset	cone slalom	gate	chair cluster	person on path	mixed clutter	crate wall gap	person and cones	barrel center
Oracle	100%	100%	100%	80%	100%	100%	100%	100%	100%	100%
Exact obstacles	100%	100%	100%	60%	80%	100%	100%	80%	100%	100%
v3, before fixes	100%	100%	60%	80%	0%	100%	100%	60%	40%	100%
v3, GT pose	100%	100%	40%	80%	70%	100%	100%	40%	60%	100%
Deployed (v5), GT pose	100%	100%	70%	80%	80%	100%	100%	20%	40%	100%
Deployed, own pose	100%	100%	50%	90%	70%	100%	100%	10%	60%	90%
Deployed, own pose, real Pi	100%	100%	80%	80%	60%	100%	100%	0%	100%	100%
Camera-only VO	100%	0%	20%	0%	0%	40%	80%	20%	0%	0%

Fig. 10. Clean success per scene (fraction of seeds). Failures concentrate on the cone slalom, the gate, the chair cluster, the crate wall and the person-and-cones scene.

$[0.047, 0.141]$). Aligned per frame, the GPU metric models score 0.20–0.22, not resolvably better than ours; raw, all overestimate distance from the 16 cm viewpoint ($\times 1.7$–3.5), plausibly from human-height training cameras. With one scale per drive MoGe-3/2 lead (0.26) and the relative networks span 0.36–0.44 (ours 0.410): their scale moves from frame to frame, so the robot takes a per-frame scale from camera height (not scored here).

Same poses, swap the depth. Integrated as a pseudo-lidar at the same independent poses, the network's range beyond 1 m is within a median 25 % of the stereo's, ray by ray (per-frame medians over 931 frames; 29 % pooled over 28,616 rays); inside 1 m both mark grass as obstacles. Both ranges come from ground-contact row geometry and the camera height, so this compares where each source places an obstacle's base, not depth values (at 3 m one pixel row is ≈ 0.12 m).

SLAM. The reference is RTAB-Map on the D435 infrared image and stereo depth: one graph over 97 % of the drive, four loop closures, 47 odometry resets. Skill ($1 - \mathrm{RMS}/\mathrm{RMS}$ of standing still) measures agreement with this reference, not accuracy: one drive; one run per method except ORB-SLAM3 stereo, DROID and VGGT (three runs each). The reference repeats to 0.33–0.79 m RMS; against any of four reference runs no skill moves by more than 0.06 and no full-coverage ranking changes. Sim(2) skill cannot fall below 0, SE(2) skill can; under Sim(2) ours scores $+0.10$ and the commands $+0.08$, $+0.02$ under the same alignment, one run: not evidence of a lead. VGGT-SLAM 2.0 reaches $+0.80$ (three bit-identical runs, 99 % of the drive); DROID-SLAM $+0.27 \pm 0.19$ (three runs; it does not rank against DPV-SLAM's $+0.39$); CoreSLAM on a RealSense scan $+0.02$; the commands alone, without vision, -0.31. ORB-SLAM3 stereo reaches $+0.879 \pm 0.013$ over three runs on the 13–19 % it tracks, but skill on a partial overlap does not rank against skill over the whole drive, and it shares the reference's infrared images. Our onboard SLAM scores -0.11 as deployed: its ground odometry's speed gate was set from the *simulated* car and rejected the fast 23 % of frames, so its path is $0.43\times$ the reference's. With a physical gate the path is $1.08\times$ (the only check of the reference's metric scale that shares none of its inputs) but the skill falls to -0.38 to -0.62, worse than the commands alone (also under Sim(2)). Uncorrected roll is the prime suspect (not isolated); an IMU heading is the planned, untested fix. As deployed, depth barely enters the trajectory: with either gate, switching the scan match off changes neither arm by more than 0.015 (physical gate: network -0.615 vs -0.618, RealSense -0.381 vs -0.382), so their gap is the odometry on different frame sets (5,612 vs 5,882), and this drive cannot rank the depths in SLAM.

Which depth is better inside SLAM? The real drive cannot say; the HITL simulator can, with ground truth. We added a D435-geometry infrared pair (SGBM), a planar lidar ring and exact depth, drove one loop twice under three lighting presets and ran every source through our SLAM (ATE after a rigid fit, mean ± sd over the three sessions). There is one seed and one trajectory, so the spread is lighting plus render nondeterminism, not seeds or routes. The student is in-distribution here: it was fine-tuned on exact disparity rendered in this same yard. The simulated car executes its commands without slip, so the commands alone reach $0.107\,\mathrm{m}$ (noon only; $0.161\,\mathrm{m}$ once two false closures from exact-depth scans are added). Every arm therefore measures how much a depth source corrupts a near-exact prior, and every camera arm except true-scale exact depth is worse than no vision. The deployed front end ignores depth: its visual-odometry prior is so tight that the scan match never moves the pose ($0.60\,\mathrm{m}$ whatever the source). With scan matching as the odometry, the deployed floor-contact pseudo-scan from *exact* depth reaches only $0.84 \pm 0.03\,\mathrm{m}$ (ours 0.65 ± 0.33). Read off the depth image's horizontal plane, exact depth reaches $0.50 \pm 0.07\,\mathrm{m}$ with the per-frame floor-fit scale. Smoothing that scale over $\sim$20 frames does not recover the loss (0.42 ± 0.13, worse at noon), while the true metric scale cuts the error 6.8× (6.3–7.4× per session), to $\mathbf{0.074 \pm 0.003}\,\mathrm{m}$. The scale error is measured: on exact depth the floor fit puts a true $2\,\mathrm{m}$ at a median 1.07× ($1\,\mathrm{m}$: 1.00×), and at 1–$2\,\mathrm{m}$ it is slow, not jittery (sd 0.32 across frames but 0.04 from one frame to the next; at $3\,\mathrm{m}$ it is heavy-tailed and jittery), so smoothing in time cannot remove the 1–$2\,\mathrm{m}$ drift. The lidar ring reaches $0.083 \pm 0.013\,\mathrm{m}$, but it is the same renderer's noise-free depth sampled on one ring: a pipeline consistency check, not evidence that camera depth equals a lidar. In our 2D SLAM on one simulated loop, the SLAM needs a better scale than the per-frame floor fit (smoothing it does not help); the per-frame scale from camera height suits avoidance, not mapping. Of the possible sources of a correct scale only stereo was tested (true-scale SGBM $0.195 \pm 0.048\,\mathrm{m}$). An inertial unit and a metric model are untested, and a single per-drive scale costs every relative Depth Anything V2- and DA3 network but one 0.15–0.32 AbsRel on the benchmark above. The same network's int8 output from the real Hailo-8L, replayed offline over the session frames, agrees with the float twin at $r = 0.9999$ per frame yet moves per-session ATE by a median $0.13\,\mathrm{m}$ (up to $0.48\,\mathrm{m}$, 12 pairs): this SLAM amplifies tiny depth differences, so only the large effects are resolved (true scale and lidar against every other arm; the commands against every camera arm without the true scale), and the sign of smoothing's effect on the network differs between the two builds.

X. Discussion

Distillation, not better quantisation. The common assumption is that a foundation model reaches a small NPU through better quantisation. On the Hailo-8L it does not: int8 attention is structurally lossy at a few hundred tokens, for our build and the vendor's. Distilling into a purely convolutional student turns a quantisation problem into a training problem, and quantisation cost then disappears from the error budget (int8 = float at $r = 0.99993$). The recipe is deliberately plain (ImageNet ResNet-34, U-Net decoder, scale-invariant loss, a rendered-ground-truth fine-tune) and transfers to any integer-only accelerator.

Metric navigation without metric depth. Ground-contact ranging asks the network only where each column's floor ends; the camera height does the rest. It is immune to the frame-to-frame scale drift of relative depth, the property that matters most on a moving robot, and its failure modes are geometric and nameable: overhangs, bases below the threshold, and surfaces filling a whole row, each with a measured detector or fix.

An evaluation that explains itself. Oracle, exact-obstacle, simulator-pose and own-pose arms decompose the gap to a perfect score into execution (0.6), perception (1.3) and localisation (0.2 clean, 0.9 reached). Without that decomposition, the interventions of Table IV would have been tuned blindly.

XI. Limitations

The suite is one reconstructed yard with static obstacles; the wheel encoder and gyro the own-pose results rely on are simulated; the student's metric range on real ground and the camera pitch are not yet calibrated; and the Pi's camera path, power draw and an autonomous drive of the real car remain to be measured (Section IX is a manual drive). The real results come from one drive, one route and one lighting condition, without ground truth. Obstacle bases below $5\,\mathrm{cm}$ are invisible to the scan, and the planner commits to routes through space the camera has not seen.

XII. Conclusion

A foundation-quality monocular depth network can run on a Raspberry Pi 5 with a Hailo-8L, but not as a transformer. Distilled into an attention-free student it runs at 27 FPS, its int8 output equals its float output, and on held-out simulated frames it is the most accurate depth network on the chip; on stereo-verified real outdoor points it is not resolvably different from the vendor's Depth Anything V2-Small. Ranged by ground-contact geometry, its relative depth becomes a metric scan without metric training, and the Pi itself maps, plans, follows and localises on it. In a real-time loop through a reconstructed yard, the system reaches the goal without contact in 7.9 of 10 scenes with simulator pose, 7.7 on its own pose and 8.2 with the real board in the loop, where the same stack with perfect obstacles reaches 9.2. The measured gap points at two concrete

TABLE V

Real drive, 1,000 stereo-verified points (park, sun; one run; D435 factory calibration). AbsRel with the scale fitted per frame and once per drive; Δ = ours (live) − method in per-frame AbsRel, frame-cluster bootstrap 95 % CI. †Scored leave-one-out on its own points in the infrared frame, holes excluded: agreement among matchers on the same images, not comparable with the monocular rows. ‡Median scale only (the others: scale and shift).

Method	AbsRel	per drive	δ_1	Δ [95 % CI]
D435 / SGBM / FFS stereo†	0.009–0.040	–	0.97–1.00	–
DA3 S–L (GPU)	0.101–0.115	0.390–0.418	0.86–0.90	+0.129 to +0.143
DAv2 S/B/L (GPU)	0.160–0.181	0.422–0.427	0.82–0.84	+0.076 to +0.092
Hailo zoo DAv2-S (chip)	0.234	0.432	0.74	+0.017 [−0.060, +0.118]
Ours, chip replay	0.258	0.413	0.74	+0.004 [+0.001, +0.006]
Ours, live on the chip	0.262	0.410	0.73	–
SC-DepthV3 (chip)‡	0.392	0.435	0.22	−0.159 [−0.242, −0.038]
FastDepth (chip)‡	0.394	0.415	0.23	−0.160 [−0.243, −0.041]
DAv2-S, our int8 (chip)	0.443	0.439	0.58	−0.178 [−0.229, −0.128]

next steps: perception of low structure and exploration that looks before it commits.

References

[1] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, "Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 3, pp. 1623–1637, 2022.

[2] L. Yang, B. Kang, Z. Huang, X. Xu, J. Feng, and H. Zhao, "Depth anything: Unleashing the power of large-scale unlabeled data," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[3] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng, and H. Zhao, "Depth anything v2," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[4] A. Bochkovskiy, A. Delaunoy, H. Germain, M. Santos, Y. Zhou, S. R. Richter, and V. Koltun, "Depth pro: Sharp monocular metric depth in less than a second," in *International Conference on Learning Representations (ICLR)*, 2025.

[5] H. Lin, S. Chen, J. H. Liew *et al.*, "Depth anything 3: Recovering the visual space from any views," *arXiv preprint arXiv:2511.10647*, 2025.

[6] Raspberry Pi Ltd, "Raspberry pi ai kit: product brief," https://datasheets.raspberrypi.com/ai-kit/raspberry-pi-ai-kit-product-brief.pdf, 2024, hailo-8L M.2 module, 13 TOPS, published June 2024.

[7] R. Ranftl, A. Bochkovskiy, and V. Koltun, "Vision transformers for dense prediction," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[8] M. Hu, W. Yin *et al.*, "Metric3d v2: A versatile monocular geometric foundation model for zero-shot metric depth and surface normal estimation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.

[9] L. Piccinelli, Y.-H. Yang, C. Sakaridis, M. Segu, S. Li, L. Van Gool, and F. Yu, "Unidepth: Universal monocular metric depth estimation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[10] X. He, D. Guo, H. Li, R. Li, Y. Cui, and C. Zhang, "Distill any depth: Distillation creates a stronger monocular depth estimator," *arXiv preprint arXiv:2502.19204*, 2025.

[11] D. Wofk, F. Ma, T.-J. Yang, S. Karaman, and V. Sze, "Fastdepth: Fast monocular depth estimation on embedded systems," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2019.

[12] N. Zhang, F. Nex, G. Vosselman, and N. Kerle, "Lite-mono: A lightweight cnn and transformer architecture for self-supervised monocular depth estimation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[13] L. Sun, J.-W. Bian *et al.*, "Sc-depthv3: Robust self-supervised monocular depth estimation for dynamic scenes," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 1, pp. 497–508, 2024.

[14] L. Ma, Y. Li, B. Jiang, Z. Zhong, H. Ding, and L. Zhu, "Asyncmde: Real-time monocular depth estimation via asynchronous spatial memory," *arXiv preprint arXiv:2603.10438*, 2026.

[15] Y. Lin, T. Zhang, P. Sun, Z. Li, and S. Zhou, "Fq-vit: Post-training quantization for fully quantized vision transformer," in *International Joint Conference on Artificial Intelligence (IJCAI)*, 2022.

[16] Z. Yuan, C. Xue, Y. Chen, Q. Wu, and G. Sun, "Ptq4vit: Post-training quantization for vision transformers with twin uniform quantization," in *European Conference on Computer Vision (ECCV)*, 2022.

[17] Z. Li, J. Xiao, L. Yang, and Q. Gu, "Repq-vit: Scale reparameterization for post-training quantization of vision transformers," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[18] N. Simon and A. Majumdar, "Mononav: Mav navigation via monocular depth estimation and reconstruction," in *International Symposium on Experimental Robotics (ISER)*, 2023.

[19] B. Sharma, P. Jadhav, P. Paul, K. M. Krishna, and A. K. Singh, "Monompc: Monocular vision based navigation with learned collision model and risk-aware model predictive control," *IEEE Robotics and Automation Letters (arXiv:2508.07387)*, 2026.

[20] J. Finke, W. P. Martis, A. Schmelter, L. Erbach, C. Jestel, and M. Wiedemann, "Learning vision-based omnidirectional navigation: A teacher-student approach using monocular depth estimation," *arXiv preprint arXiv:2603.01999*, 2026.

[21] D. Shah, A. Sridhar, N. Dashora, K. Stachowicz, K. Black, N. Hirose, and S. Levine, "Vint: A foundation model for visual navigation," in *Conference on Robot Learning (CoRL)*, 2023.

[22] A. Sridhar, D. Shah, C. Glossop, and S. Levine, "Nomad: Goal masked diffusion policies for navigation and exploration," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.

[23] F. Yang, C. Wang, C. Cadena, and M. Hutter, "iplanner: Imperative path planning," in *Robotics: Science and Systems (RSS)*, 2023.

[24] P. Roth, J. Nubert, F. Yang, M. Mittal, and M. Hutter, "Viplanner: Visual semantic imperative learning for local navigation," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.

[25] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel, "Path planning for autonomous vehicles in unknown semi-structured environments," *The International Journal of Robotics Research*, vol. 29, no. 5, pp. 485–501, 2010.

[26] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Carnegie Mellon University, Robotics Institute, Tech. Rep. CMU-RI-TR-92-01, 1992.

[27] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[28] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015.

[29] B. D. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision," in *International Joint Conference on Artificial Intelligence (IJCAI)*, 1981.

[30] NVIDIA *et al.*, "Isaac lab: A gpu-accelerated simulation framework for multi-modal robot learning," *arXiv preprint arXiv:2511.04831*, 2025.

[31] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics*, vol. 42, no. 4, 2023.

[32] Q. Wu, J. Martinez Esturo, A. Mirzaei, N. Moenne-Loccoz, and Z. Gojcic, "3dgut: Enabling distorted cameras and secondary rays in gaussian splatting," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.